

Question 1:What is a function in Python? Explain how functions are created and called. Also explain parameters, arguments, default parameters, *args, and **kwargs with suitable examples.

Answer :- A **function** in Python is a reusable block of organized code designed to perform a specific, single task. Functions promote modularity, code reusability, and easier debugging by breaking complex programs into manageable parts.

Creating and Calling Functions

- **Creating a Function:** Defined using the `def` keyword, followed by a function name, parentheses `()`, and a colon `:`. The code block inside is indented.
- **Calling a Function:** Executed by referencing its name followed by parentheses `()`.

Python

Function Definition

```
def greet():
```

```
    print("Hello, welcome to Python!")
```

Function Call

```
greet()
```

Parameters vs. Arguments

- **Parameters:** Variables listed inside the parentheses in the function definition (acting as placeholders).
- **Arguments:** The actual values passed to the function when it is called.

Python

'name' is a parameter

```
def greet_user(name):
```

```
    print(f"Hello, {name}!")
```

"Alice" is an argument

```
greet_user("Alice")
```

Default Parameters

A default parameter assumes a default value if no argument is provided during the function call.

Python

```
def greet_person(name="Guest"):
```

```
    print(f>Welcome, {name}!")
```

```
greet_person("Bob") # Uses provided argument -> Welcome, Bob!
```

```
greet_person()      # Uses default value    -> Welcome, Guest!
```

***args** (Positional Variable-Length Arguments)

***args** allows a function to accept any number of positional arguments. Inside the function, **args** is received as a **tuple**.

Python

```
def sum_numbers(*args):
```

```
    total = sum(args)
```

```
    print(f"Sum: {total}")
```

```
sum_numbers(10, 20)      # Sum: 30
```

```
sum_numbers(5, 10, 15, 20) # Sum: 50
```

****kwargs** (Keyword Variable-Length Arguments)

****kwargs** allows a function to accept any number of keyword (named) arguments. Inside the function, **kwargs** is received as a **dictionary**.

Python

```
def display_profile(**kwargs):
```

```
    for key, value in kwargs.items():
```

```
        print(f"{key}: {value}")
```

```
display_profile(name="Sarah", age=25, city="New York")
```

Output:

name: Sarah

age: 25

city: New York

Question 2: What is variable scope in Python? Explain local and global variables. Why should excessive use of the global keyword be avoided? Also explain nested functions and closures

Answer :- **Variable scope** refers to the region or context in Python program code where a variable is recognized and accessible. Scope determines where a variable can be created, accessed, and modified within a script.

Local vs. Global Variables

- **Local Variables:** Defined inside a function or block. They can only be accessed within that specific function and are destroyed once execution finishes.
- **Global Variables:** Defined outside any function (usually at the top level of the script). They are accessible from anywhere within the file.

Python

```
global_var = "I am Global" # Global variable
```

```
def test_scope():
```

```
    local_var = "I am Local" # Local variable
```

```
    print(local_var)
```

```
    print(global_var)
```

```
test_scope()
```

```
# print(local_var) # Error: NameError: name 'local_var' is not defined
```

Why Excessive Use of the **global** Keyword Should Be Avoided

The **global** keyword allows you to modify a global variable inside a local scope (a function). However, using it excessively is considered bad practice because:

1. **Loss of Predictability & Debugging Difficulty:** Any function can modify global state unpredictably, making side effects hard to trace.
2. **Tight Coupling:** Functions become tightly coupled to global state, making code reuse and refactoring difficult.
3. **Concurrency Issues:** In multithreaded or complex applications, simultaneous modification of global state leads to race conditions.

Nested Functions

A **nested function** (or inner function) is a function defined inside another function (outer function). The inner function can access variables defined in the outer function's scope.

Python

```
def outer_function():
```

```
outer_var = "Outer Scope Variable"
```

```
def inner_function():
```

```
    print(f"Accessing from inner: {outer_var}")
```

```
inner_function()
```

```
outer_function()
```

Closures

A **closure** is a nested function that retains access to variables from its enclosing (outer) scope, even after the outer function has finished executing and returned.

For a closure to exist, three conditions must be met:

1. There must be a nested (inner) function.
2. The inner function must reference a variable defined in the outer function.
3. The outer function must return the inner function.

Python

```
def make_multiplier(x):
```

```
# Outer function

def multiply(y):

    # Inner function accessing variable 'x' from outer scope

    return x * y

return multiply # Returns the inner function without calling it


# 'double' is a closure that remembers x = 2

double = make_multiplier(2)


print(double(5)) # Output: 10

print(double(10)) # Output: 20
```

Question 3: What is recursion in Python? Explain how a recursive function works. What is a base condition, and why is it important? Explain recursion using the factorial of a number as an example.

Answer :- **Recursion** is a programming technique in Python where a function calls itself directly or indirectly to break down a problem into smaller, more manageable sub-problems until a simple terminating condition is met.

How a Recursive Function Works

A recursive function works by dividing a task into two main components:

1. **The Recursive Step:** The function calls itself with a smaller or simpler input, progressing toward a stopping condition.
 2. **The Call Stack Execution:** Each recursive call is pushed onto the system's call stack. Python pauses execution of the outer function until the inner function returns a result. Once the base condition is reached, the results unwind back through the call stack to compute the final answer.
-

What is a Base Condition and Why is it Important?

- **Base Condition:** The terminating scenario in a recursive function that stops further recursive calls and returns a direct value.
 - **Importance:** Without a base condition, a recursive function would call itself indefinitely, eventually exceeding Python's maximum recursion depth and causing a **RecursionError: maximum recursion depth exceeded** (often referred to as stack overflow).
-

Explaining Recursion Using Factorial

The factorial of a non-negative integer n (denoted as $n!$) is the product of all positive integers less than or equal to n .

Mathematically:

- **Base case:** $0! = 1$ and $1! = 1$
- **Recursive step:** $n! = n \times (n - 1)!$

Python Implementation

Python

```
def factorial(n):
```

```
    # Base Condition
```

```
    if n == 0 or n == 1:
```

```
        return 1
```

```
    # Recursive Call
```

```
    else:
```

```
        return n * factorial(n - 1)
```

```
# Example Usage
```

```
number = 5
```

```
print(f"The factorial of {number} is {factorial(number)}") # Output: 120
```

Step-by-Step Execution for `factorial(4)`

1. `factorial(4)` calls `4 * factorial(3)`
2. `factorial(3)` calls `3 * factorial(2)`
3. `factorial(2)` calls `2 * factorial(1)`
4. `factorial(1)` hits the **base condition** and returns 1.

Now, the execution unwinds back up the stack:

- `factorial(2)` computes `2 * 1 = 2` and returns 2
- `factorial(3)` computes `3 * 2 = 6` and returns 6

- `factorial(4)` computes $4 * 6 = 24$ and returns **24**

Question 4:What is the difference between an iterable, an iterator, and a generator in Python? Explain the purpose of `iter()`, `next()`, `StopIteration`, and `yield`

Answer :- Iterable vs. Iterator vs. Generator

Feature	Iterable	Iterator	Generator
Definition	An object capable of returning its members one at a time (e.g., list , tuple , string).	An object representing a stream of data; remembers its state during iteration.	A special type of iterator created using a function with yield .
Methods Required	Implements <code>__iter__()</code> .	Implements both <code>__iter__()</code> and <code>__next__()</code> (Iterator Protocol).	Implements <code>__iter__()</code> and <code>__next__()</code> automatically.

Memory Efficiency	Stores all elements in memory at once.	Processes elements on-demand (lazy evaluation).	Processes elements on-demand without storing the whole sequence in memory.
Creation	Sequences, collections, or custom classes with <code>__iter__()</code> .	By calling <code>iter()</code> on an iterable or defining a custom class.	Defined using functions containing the <code>yield</code> statement.

Purpose of Core Iteration Keywords & Functions

1. `iter()`

- **Purpose:** Converts an **iterable** into an **iterator**.
- **How it works:** It calls the object's internal `__iter__()` method.

Python

```
numbers = [10, 20, 30] # Iterable
```

```
iterator_obj = iter(numbers) # Iterator
```

2. `next()`

- **Purpose:** Retrieves the **next item** from an iterator.
- **How it works:** Calls the iterator's `__next__()` method and advances the internal state.

Python

```
print(next(iterator_obj)) # Output: 10
```

```
print(next(iterator_obj)) # Output: 20
```

3. `StopIteration`

- **Purpose:** A built-in exception raised to signal that there are **no more items** in an iterator.
- **How it works:** When `next()` is called on an exhausted iterator, `StopIteration` is raised, causing loops (like `for` loops) to terminate gracefully.

Python

```
print(next(iterator_obj)) # Output: 30
```

```
# print(next(iterator_obj)) # Raises StopIteration exception
```

4. `yield`

- **Purpose:** Used inside a function to turn it into a **generator**.
- **How it works:** Unlike **return** (which terminates a function and returns a single value), **yield** pauses function execution, saves its state, and produces a value to the caller. When **next()** is called again, execution resumes right after the **yield** statement.

Python

```
def count_up_to(limit):
```

```
    count = 1
```

```
    while count <= limit:
```

```
        yield count
```

```
        count += 1
```

```
# Using the generator
```

```
counter = count_up_to(3)
```

```
print(next(counter)) # Output: 1
```

```
print(next(counter)) # Output: 2
```

```
print(next(counter)) # Output: 3
```

Question 5: What is a lambda function? Explain how map(), filter(), and reduce() work in Python with suitable examples.

Answer :- A **lambda function** in Python is a small, anonymous function defined without a name using the **lambda** keyword. Unlike standard functions defined with **def**, lambda functions are written in a single line, can take any number of arguments, but can only evaluate and return a **single expression**.

Syntax:

Python

lambda arguments: expression

Example:

Python

Standard function

```
def add_five(x):
```

```
    return x + 5
```

Equivalent lambda function

```
add_five_lambda = lambda x: x + 5
```

```
print(add_five_lambda(10)) # Output: 15
```

Higher-Order Functional Concepts

1. `map()` Function

- **Purpose:** Applies a given function to **every item** in an iterable (like a list) and returns an iterator with the transformed results.
- **Syntax:** `map(function, iterable)`

Python

```
numbers = [1, 2, 3, 4, 5]
```

```
# Using map() with lambda to double each number
```

```
doubled = list(map(lambda x: x * 2, numbers))
```

```
print(doubled) # Output: [2, 4, 6, 8, 10]
```

2. `filter()` Function

- **Purpose:** Filters elements from an iterable based on whether a function returns `True` or `False` for each element.
- **Syntax:** `filter(function, iterable)`

Python

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
```

```
# Using filter() with lambda to keep only even numbers
```

```
evens = list(filter(lambda x: x % 2 == 0, numbers))
```

```
print(evens) # Output: [2, 4, 6, 8]
```

3. `reduce()` Function

- **Purpose:** Performs a cumulative operation on pairs of elements in an iterable, reducing the entire sequence to a **single cumulative value**.
- **Note:** Must be imported from the `functools` module.
- **Syntax:** `reduce(function, iterable)`

Python

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4, 5]
```

```
# Using reduce() with lambda to calculate the product of all elements
```

```
product = reduce(lambda x, y: x * y, numbers)
```

```
print(product) # Output: 120 (Calculation: (((1*2)*3)*4)*5)
```

Question 6: Write a Python function called `calculate_total()` that:

1. Accepts any number of marks using `*args`.
2. Calculates the total marks.
3. Uses a default bonus value of 5.
4. Adds the bonus to the total.
5. Returns the final result

Here is the Python implementation for `calculate_total()` satisfying all the requirements:

Python

```
def calculate_total(*args, bonus=5):
```

```
    """
```

```
    Calculates the total marks from variable positional arguments
    and adds a default bonus value.
```

```
    """
```

```
    # 1. Calculates the sum of all marks passed via *args
```

```
    total_marks = sum(args)
```

2. Adds the bonus to the total

```
final_total = total_marks + bonus
```

3. Returns the final result

```
return final_total
```

Example Usage:

Passing marks with the default bonus of 5

```
result1 = calculate_total(80, 85, 90)
```

```
print(f"Total with default bonus (5): {result1}") # Output: 260 (80 + 85  
+ 90 + 5)
```

Passing marks and overriding the default bonus

```
result2 = calculate_total(80, 85, 90, bonus=10)
```

```
print(f"Total with custom bonus (10): {result2}") # Output:
```

Question 7: Write a recursive Python function to reverse a string. Use the string: "Python" Display both the original and reversed strings...

Python

```
def reverse_string(s):
```

```
    # Base Condition: An empty string or single-character string is  
    already reversed
```

```
    if len(s) <= 1:
```

```
        return s
```

```
    # Recursive Case: Last character + reverse of the remaining slice
```

```
    return s[-1] + reverse_string(s[:-1])
```

```
# Input string
```

```
original_string = "Python"
```

```
# Call the recursive function
```

```
reversed_string = reverse_string(original_string)
```

```
# Display both the original and reversed strings
```

```
print(f"Original String: {original_string}")
```

```
print(f"Reversed String: {reversed_string}")
```

Output:

Plaintext

Original String: Python

Reversed String: nohtyP

Question 8: Create a custom iterator class called EvenNumbers that generates even numbers from 2 to 10. Use `__iter__()`, `__next__()`, and `StopIteration`.

Answer :- Python

```
class EvenNumbers:
```

```
    def __init__(self):
```

```
        # Start at 2 (the first even number in the range)
```

```
        self.current = 2
```

```
    def __iter__(self):
```

```
        # Returns the iterator object itself
```

```
        return self
```

```
    def __next__(self):
```

```
        # Stop iteration once we exceed 10
```

```
        if self.current > 10:
```

```
            raise StopIteration
```

```
        # Save current even number and increment by 2 for the next call
```

```
        val = self.current
```

```
        self.current += 2
```

```
return val
```

```
# Demonstration / Usage
```

```
even_iter = EvenNumbers()
```

```
# Using a for loop to iterate through the custom iterator
```

```
for num in even_iter:
```

```
    print(num)
```

Output:

Plaintext

2

4

6

8

10

Question 9: Write a generator function that accepts a number n and generates all even numbers from 1 to n. Use yield instead of returning a list.

Answer :- Python

```
def generate_evens(n):
```

```
    """
```

```
    Generator function that yields all even numbers from 1 to n.
```

```
    """
```

```
    for i in range(1, n + 1):
```

```
        if i % 2 == 0:
```

```
            yield i
```

```
# Example Usage:
```

```
n = 10
```

```
print(f"Even numbers from 1 to {n}:")
```

```
# Iterating over the generator
```

```
for num in generate_evens(n):
```

```
print(num)
```

Output:

Plaintext

Even numbers from 1 to 10:

2

4

6

8

10

Question 10: Consider the following list:

```
numbers = [1, 2, 3, 4, 5, 6]
```

Write a Python program to:

1. Use `map()` to calculate the square of every number.
2. Use `filter()` to select only even numbers.
3. Use `reduce()` to calculate the sum of all numbers.
4. Display all three results.

Answer :- Python

```
from functools import reduce
```

```
# Input list
```

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
# 1. Use map() to calculate the square of every number
```

```
squared_numbers = list(map(lambda x: x ** 2, numbers))
```

```
# 2. Use filter() to select only even numbers
```

```
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
```

```
# 3. Use reduce() to calculate the sum of all numbers
```

```
total_sum = reduce(lambda x, y: x + y, numbers)
```

```
# 4. Display all three results
```

```
print(f"Original Numbers: {numbers}")
```

```
print(f"Squared Numbers (map): {squared_numbers}")
```

```
print(f"Even Numbers (filter): {even_numbers}")
```

```
print(f"Sum of Numbers (reduce): {total_sum}")
```

Output:

Plaintext

Original Numbers: [1, 2, 3, 4, 5, 6]

Squared Numbers (map): [1, 4, 9, 16, 25, 36]

Even Numbers (filter): [2, 4, 6]

Sum of Numbers (reduce): 21